ExamGuard AI

> **AI-Powered Intelligent Online Exam Invigilation Platform**

Building trust in online examinations using advanced AI physical perception, real-time monitoring, and intelligent risk analysis.

Overview

ExamGuard AI is a comprehensive SaaS platform that ensures the integrity of online examinations through:

- **Physical Perception**: Real-time webcam and microphone analysis using Afferens Vision API
- **AI Reasoning Engine**: Context-aware intelligence that understands student behavior
- **Intelligent Action**: Automatic warnings, evidence capture, and risk assessment
- **Live Monitoring**: Real-time teacher dashboard with risk scores and alerts
- **Comprehensive Reports**: Automatic PDF generation with AI summaries

Architecture

```
Physical World
      ↓
Webcam + Microphone
      ↓
Afferens Vision API
      ↓
AI Reasoning Engine
      ↓
Risk Analysis
      ↓
Decision
      ↓
Warning / Alert
      ↓
Teacher Dashboard
      ↓
Final AI Report
```

Tech Stack

Frontend

- **Next.js 14** - React framework with App Router
- **TypeScript** - Type-safe development
- **Tailwind CSS** - Utility-first styling
- **Framer Motion** - Animations and transitions
- **Zustand** - State management
- **Recharts** - Data visualization
- **Supabase Auth** - Authentication

Backend

- **FastAPI** - High-performance Python API framework
- **Python 3.11** - Modern Python
- **Supabase PostgreSQL** - Database
- **WebSockets** - Real-time communication
- **OpenAI GPT-4o** - AI reasoning and explanations
- **Afferens Vision API** - Computer vision analysis

Infrastructure

- **Vercel** - Frontend deployment
- **Docker** - Containerization
- **Redis** - Caching and session storage

Features

For Teachers

- Create and schedule exams with auto-generated codes
- Live monitoring dashboard with webcam grid
- Real-time risk scores and alerts
- Clickable event timeline
- Automatic evidence gallery
- Comprehensive PDF reports
- Student management

For Students

- Join exams with 6-digit code or secure link
- Identity verification with webcam
- Environment scanning
- Secure exam interface with fullscreen enforcement
- Real-time AI monitoring with warnings

For Admins

- Manage teachers and students
- Institute management
- System analytics
- Subscription management
- Audit logs

AI Detection Capabilities

- Face detection and verification
- Multiple face detection
- Phone detection
- Book/notebook detection
- Calculator detection
- Extra monitor detection

- Earphones detection
- Head pose analysis
- Eye movement tracking
- Looking away detection
- Camera coverage detection
- Poor lighting detection
- Background change detection
- Desk object detection
- Empty chair detection
- Student left detection
- Audio analysis (talking, whispering, background conversation)

Project Structure

```
examguard-ai/
├── frontend/                 # Next.js frontend
│   ├── src/
│   │   ├── app/              # App Router pages
│   │   ├── components/       # React components
│   │   │   ├── ui/           # shadcn-style UI components
│   │   │   ├── dashboard/    # Dashboard components
│   │   │   ├── exam/         # Exam components
│   │   │   └── landing/      # Landing page components
│   │   ├── hooks/            # Custom React hooks
│   │   ├── lib/              # Utilities and API client
│   │   ├── store/            # Zustand stores
│   │   └── types/            # TypeScript types
│   ├── package.json
│   └── next.config.js
│
├── backend/                  # FastAPI backend
│   ├── app/
│   │   ├── api/v1/endpoints/  # API routes
│   │   ├── core/              # Config, security, middleware
│   │   ├── db/                # Database layer
│   │   ├── services/          # Business logic
│   │   │   ├── afferens.py    # Afferens API integration
│   │   │   ├── ai_engine.py   # AI reasoning engine
│   │   │   ├── risk_analyzer.py
│   │   │   ├── evidence.py
│   │   │   └── report_generator.py
│   │   └── websocket/         # WebSocket handlers
│   ├── migrations/            # Database migrations
│   ├── tests/                 # Test suite
│   ├── Dockerfile
│   └── requirements.txt
│
├── infra/                    # Infrastructure
│   ├── docker/
│   ├── terraform/
```

```
|   └── k8s/
|
└── docs/                    # Documentation
```

Quick Start

Prerequisites

- Node.js 20+
- Python 3.11+
- Docker (optional)
- Supabase account
- Afferens API key
- OpenAI API key

1. Clone and Setup

```
git clone https://github.com/your-org/examguard-ai.git
cd examguard-ai
```

2. Environment Configuration

```
cp .env.example .env
# Edit .env with your credentials
```

3. Database Setup

Run the migration in Supabase SQL Editor:

```
# Execute backend/migrations/001_initial_schema.sql in Supabase
```

4. Backend Setup

```
cd backend
python -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
pip install -r requirements.txt

# Run development server
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

5. Frontend Setup

```
cd frontend
npm install

# Run development server
npm run dev
```

6. Docker Setup (Alternative)

```
docker-compose -f infra/docker/docker-compose.yml up --build
```

API Documentation

Once the backend is running, visit:

- Swagger UI: `http://localhost:8000/api/docs`
- ReDoc: `http://localhost:8000/api/redoc`

Key API Endpoints

Authentication

- `POST /api/v1/auth/login` - Login with email/password
- `POST /api/v1/auth/refresh` - Refresh access token
- `GET /api/v1/auth/me` - Get current user

Exams

- `POST /api/v1/exams` - Create exam
- `GET /api/v1/exams` - List exams
- `GET /api/v1/exams/{id}` - Get exam details
- `GET /api/v1/exams/code/{code}` - Get exam by code (public)

Sessions

- `POST /api/v1/sessions/join` - Student joins exam
- `GET /api/v1/sessions/{id}` - Get session
- `POST /api/v1/sessions/{id}/submit` - Submit exam

AI Analysis

- `POST /api/v1/afferens/analyze` - Analyze video frame
- `POST /api/v1/afferens/analyze-audio` - Analyze audio

Dashboard

- `GET /api/v1/dashboard/stats` - Dashboard statistics
- `GET /api/v1/dashboard/live-sessions` - Live monitoring sessions

WebSocket Protocol

Student Connection

```
const ws = new WebSocket('ws://localhost:8000/ws/monitor/{session_id}');

// Send frame for analysis
ws.send(JSON.stringify({
  type: 'frame',
  frame: 'base64_encoded_image'
}));
```

```
// Receive analysis results
ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  // { type: 'frame_analysis', payload: { risk_score, detections, ... } }
};
```

Teacher Dashboard Connection

```
const ws = new WebSocket('ws://localhost:8000/ws/teacher/{teacher_id}');

// Send warning to student
ws.send(JSON.stringify({
  type: 'send_warning',
  session_id: '...',
  message: 'Please face the camera'
}));
```

AI Reasoning Engine

The AI Reasoning Engine uses OpenAI GPT-4o to provide context-aware analysis:

Example Decision Logic:

- Phone visible for 1 second → **Ignore** (notification)
- Phone visible for 40 seconds → **High Risk** (likely cheating)
- Student drinks water → **Ignore** (normal behavior)
- Student continuously looking left → **Suspicious** (might be reading notes)
- Student leaves room → **High Risk** (unauthorized absence)

Every decision includes a natural language explanation.

Risk Score System

Score	Level	Color	Action
0-25	Safe	Green	Continue monitoring
26-50	Attention	Yellow	Monitor closely
51-75	Suspicious	Orange	Issue warning
76-100	High Risk	Red	Alert teacher / Auto-terminate

Security Features

- JWT-based authentication with refresh tokens
- Role-based access control (RBAC)
- Encrypted storage
- Signed URLs for evidence
- Rate limiting

- Audit logs
- Session timeouts
- Fullscreen enforcement
- Tab switching detection
- Copy-paste prevention
- Right-click blocking
- Keyboard shortcut blocking

Deployment

Frontend (Vercel)

```
cd frontend
vercel --prod
```

Backend (Docker)

```
cd backend
docker build -t examguard-backend .
docker run -p 8000:8000 --env-file ../.env examguard-backend
```

Environment Variables

Variable	Description	Required
`SUPABASE_URL`	Supabase project URL	Yes
`SUPABASE_SERVICE_KEY`	Supabase service role key	Yes
`AFFERENS_API_KEY`	Afferens Vision API key	Yes
`OPENAI_API_KEY`	OpenAI API key	Yes
`SECRET_KEY`	JWT secret key	Yes

Testing

```
# Backend tests
cd backend
pytest

# Frontend tests
cd frontend
npm test
```

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit changes (`git commit -m 'Add amazing feature'`)
4. Push to branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

License

MIT License - see LICENSE file for details.

Support

- Documentation: docs.examguard.ai
- API Reference: api.examguard.ai
- Email: support@examguard.ai

Built with ❤️ for secure online examinations.